



Total Points

27 / 30 pts

Question 1

Efficient declarative algorithms

10 / 10 pts

1.1 Amortized time complexity

2 / 2 pts

✓ - 0 pts Correct

- 2 pts Incorrect

- 0.5 pts Error in formula

- 1 pt Missing formula

- 0.5 pts Error in explanation

- 1 pt Incorrect explanation

1.2 Ephemeral and persistent algorithm

2 / 2 pts

✓ - 0 pts Correct

- 2 pts Incorrect

- 0.5 pts Mostly correct but with a few errors

- 1 pt Error in explanation

1.3 Amortized constant-time ephemeral queue

3 / 3 pts

✓ - 0 pts Correct

- 3 pts Incorrect

- 2 pts Mostly incorrect, small part correct

- 1 pt Mostly correct, some error

1.4 Worst-case constant-time ephemeral queue

3 / 3 pts

✓ - 0 pts Correct

- 3 pts Incorrect

- 1.5 pts Errors in difference list

- 1 pt Some errors

Question 2

One_for_one supervision

2 / 2 pts

✓ - 0 pts Correct

- 2 pts Incorrect
- 0.5 pts Practical scenario missing
- 0.5 pts Definition of supervisor tree missing

Question 3

FoldL and port objects

1 / 3 pts

- 0 pts Correct
- 3 pts Incorrect
- 0.5 pts Error in FoldL
- 1 pt Missing FoldL definition
- 0.5 pts Small error in port object creation
- 1 pt Error in port object creation

✓ - 2 pts Missing port object definition

Question 4

Erlang's generic server

5 / 5 pts

✓ - 0 pts Correct

- 5 pts Incorrect
- 1 pt Missing functionality of specific module
- 1 pt Missing functionality of generic module
- 1 pt Missing external interface of specific module
- 1 pt Missing external interface of generic module
- 1 pt Missing higher-order programming or message passing (callback interface, how modules are connected)

Question 5

Reentrant locks

4 / 5 pts

– 0 pts Correct

– 5 pts Incorrect

✓ – 1 pt Specification missing

– 1 pt Importance not explained correctly

– 1 pt Error(s) in implementation

– 2 pts Implementation missing or mostly wrong

– 1 pt Token passing not explained

Question 6

Safety and liveness

5 / 5 pts

✓ – 0 pts Correct

– 5 pts Incorrect

– 0.5 pts Incomplete definition(s) of execution, prefix, extension.

– 1 pt Incorrect definition(s)

– 0.5 pts Incomplete definition of safety property

– 1 pt Incorrect definition of safety

– 0.5 pts Incomplete definition of liveness property

– 1 pt Incorrect definition of liveness

– 1 pt No example of safety property

– 1 pt No example of liveness property

– 1 pt Problem with example(s)

Final exam LINFO1131

August 16, 2024
Prof. Peter Van Roy

Please write all answers inside the rectangles

First and last	
NOMA	

Q1: Efficient declarative algorithms [10pts] (Q1 corresponds to the midterm)

Q1.1: Define the concept of *amortized time complexity*. Assume big-O notation is known. [2pts]

Si l'on sait que m opérations ont une complexité globale en $O(f(m))$, alors on peut dire que chaque opération a une complexité amortie de $O(f(m)/m)$.
⇒ C'est utile lorsque les opérations individuelles sont coûteuses mais l'algorithme en général bon marché.

Q1.2: Define the concepts of an *ephemeral algorithm* and a *persistent algorithm*. [2pts]

Soit une Queue Q_1 . On effectue l'opération suivante : $Q_2 = \{\text{Insert } Q_1 \text{ a}\}$
⇒ Dans un algorithme éphémère, l'ancienne version de la Queue Q_1 n'est plus disponible et ne peut plus être utilisée. De nombreux langages de programmation (ex: Python) utilisent cette manière de fonctionner. Il est possible de garder une version de Q_1 mais c'est entièrement géré par le programmeur (en copiant).
⇒ Dans un algorithme persistant, l'ancienne version de la Queue Q_1 est toujours disponible et peut toujours être utilisée. C'est pratique notamment pour le travail collaboratif (si 2 personnes modifient la même Queue en même temps).

Q1.3: Give the code of an amortized constant-time ephemeral queue in sequential functional programming (no single assignment and no lazy evaluation). [3pts]

```

fun {NewQueue} q (nil nil) and
  fun {Check Q}
    case Q of q (nil R) then q ([Reverse R] nil) else Q and
  and
  fun {Insert Q X}
    case Q of q (F R) then {Check q (F X R)} end
  and
  fun {Delete Q X}
    case Q of q (F R) then F1 in
      F = X1F1 {Check q (F1 R)} and
    end
  end
end

```

Q1.4: Give the code of a worst-case constant-time ephemeral queue in sequential functional programming with single assignment (no lazy evaluation). [3pts]

```

fun {NewQueue} X in q (X X) and
  fun {Insert Q X}
    case Q of q (S E) then E1 in
      E = X1E1 q (S E1) and
    end
  and
  fun {Delete Q X}
    case Q of q (S E) then S1 in
      S = X1S1 q (S1 E) and
    end
  end
end

```

Q2: One_for_one supervision [2pts]

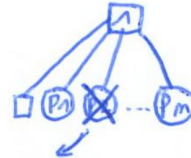
Explain how one_for_one supervision works in Erlang and give a practical scenario of its use. As part of your answer, define the concept of "supervisor tree".

Un Arbre de Supervision est constitué d'un ensemble de modules de supervision, organisé sous forme d'arbre avec des modules internes et un module racine. Chaque module superviseur est responsable du démarrage, de l'arrêt et de la surveillance de ses modules enfants.

La Stratégie one-for-one est une stratégie de supervision où lorsqu'un enfant meurt, c'est le seul à être redémarré.

On utilise cette stratégie dans le cas où les modules enfants sont indépendants les uns des autres.

Stratégie One-for-one :



P2 est le seul à être redémarré

Q3: FoldL and port objects [3 pts]

Give the Oz code for the FoldL operation and use it to define a port object.

```

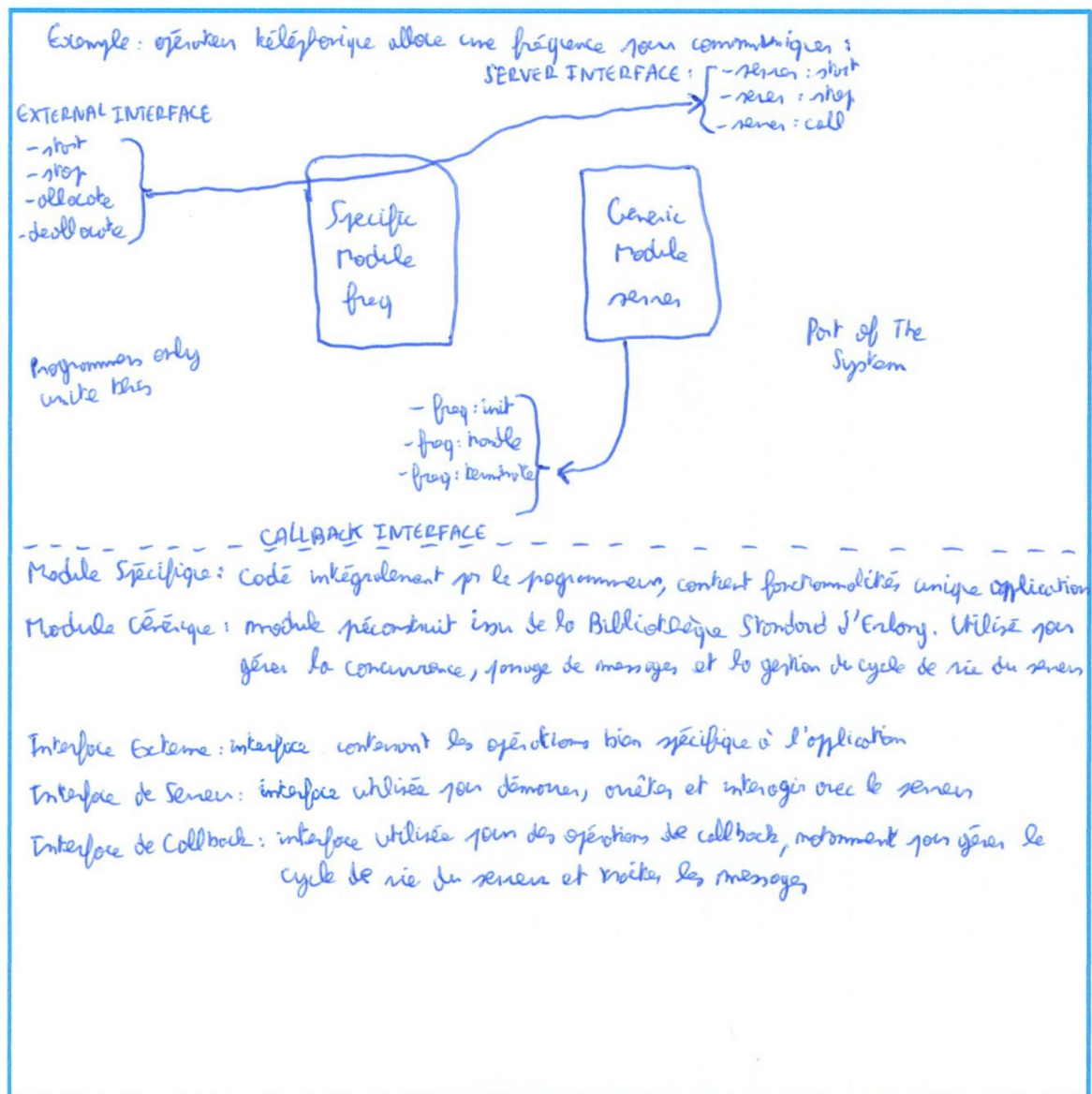
fun {FoldL L F U}
  case L of HIT then
    {FoldL T F {F U H3}}
  [] nil then nil
  end
end
end

```

↳ $p = \{ \text{FoldL } S \text{ nil nil} \}$

Q4: Erlang's generic server [5pts]

To make it easier to build concurrent and fault-tolerant applications, Erlang provides a set of generic components called "behaviors". One of the most-used Erlang behaviors is the generic server. In an application using a generic server, there will be two modules, a specific module and a generic module. For this question, give a simple example of an application that uses the generic server (show it as a block diagram). Answer the following questions. What is the functionality of the specific module (what are its operations)? What is the functionality of the generic module (what are its operations)? What does the external interface of the specific module look like? What does the external interface of the generic module look like? How are the specific and generic modules connected together (what programming technique is used)?



Q5: Reentrant locks [5pts]

For this question, answer the following four points:

1. Specify a reentrant lock.
2. Explain why reentrancy is important. Give a scenario where it is needed.
3. Give an implementation of the reentrant lock, similar to what we saw in the course.
4. Explain how the implementation works by showing how it does token passing.

1) Un Reentrant lock est comme un lock classique (= concept qui garantit qu'un seul thread à la fois a dans une partie de programme appelée section critique), sauf qu'il a la particularité qu'un même thread peut entrer, sortir et réentrer dans la même section critique plusieurs fois d'affilée.

2) La réentrance est pratique si on veut par exemple faire 2 insert de suite dans une Queue ou que l'on veut éviter les Deadlocks.

```

3)
fun {NewLock}
  Token = {NewCell unit}
  CurTh = {NewCell unit}
  proc {Lock P}
    if {Thread.kthis} == @CurTh then
      {P}
    else OldNew in
      {Exchange Token OldNew}
      {Unit Old}
      CurTh := {Thread.kthis}
    try {P} finally
      CurTh := unit
      New = unit
    end
  end
end
in 'lock'('lock': Lock)
end

```

| L'implémentation fonctionne comme suit :

| Le code va regarder si le thread qui essaie de rentrer dans le nouveau est le même que le dernier qui est rentré dedans. Si c'est le cas, alors le code donne accès à la section critique {P}.

| Si ce n'est pas le cas, un nouveau jeton va s'organiser avec l'opération "Exchange" entre le nouveau et l'ancien thread. On attend ensuite que l'ancien thread termine ce qu'il a à faire et puis le nouveau thread, qui a maintenant le jeton, peut rentrer dans la section critique. On change le contenu de la cellule CurTh par le nouveau thread.

Q6: Safety and liveness [5pts]

For this question, answer the following four points:

1. Give formal definitions of execution, prefix of an execution, and extension of a prefix.
2. Give a formal definition of a safety property.
3. Give a formal definition of a liveness property.
4. Give an example of a safety property and an example of a liveness property.

1)

- Une Exécution est une séquence d'états d'exécution $(\sigma_0, \sigma_0) \rightarrow \dots (\sigma_m, \sigma_m)$
- Un Préfixe d'une exécution est constitué des k ($k > 0$) premiers états d'exécution
- Une Extension d'un préfixe est une exécution commençant par un préfixe

2)

SAFETY:

Une propriété est dite Safety si étant donné une exécution E telle que $P(E) = \text{False}$, alors il existe un préfixe P de E tel que toute extension de P donne une exécution F avec $P(F) = \text{False}$.

3)

LIVENESS:

Une propriété est dite Liveness si étant donné toute exécution E telle que $P(E) = \text{True}$, alors pour tout préfixe F de E , il existe une extension de F telle que $P(F) = \text{True}$.

4)

Exemple: communication de messages sur un réseau par fibres

→ Safety: "un message est délivré au plus une fois"

→ Liveness: "un message est délivré au moins une fois"